



BOTSWANA INTERNATIONAL UNIVERSITY
OF SCIENCE & TECHNOLOGY

MODULE NAME: MICROCONTROLLERS

MODULE CODE: CTEN 415

DEPARTMENT: Electrical, Computer and Telecommunications Engineering

MODULE INSTRUCTOR: JWAONE GABABOITAOLELWE

TITLE: DIGITAL VOTING SYSTEM

FULL NAME	STUDENT ID	COURSE
TSOTLHE SEIPHEPI	21001137	BEng Computer and Telecommunications
KGOTSO MATHANGWANE	21000459	BEng Electrical and Electronics

INTRODUCTION

This report details the design and implementation of a digital voting system using a PIC16F877A microcontroller. The system is designed to provide a user-friendly and secure platform for casting votes, with a 16x2 LCD to display candidate information and a 4x4 matrix keypad allowing voters to input their choices.

THEORY

A microcontroller (MCU) is a compact computer on a single integrated circuit, designed to control specific tasks within electronic systems. It integrates a CPU, memory, and input/output interfaces on one chip. Microcontrollers are widely used in embedded systems, such as home appliances, automotive systems, medical devices, and consumer electronics.[1]

A typical microcontroller includes a processor core (for executing instructions), volatile and non-volatile memory (for data and program storage), and I/O peripherals (for interacting with the external environment). They also support various communication interfaces.

Microcontrollers are programmable, allowing them to be customized for specific tasks. Common programming languages include C, C++, and assembly language. Designed for low-power operation, they are ideal for battery-operated devices and are embedded in a wide range of products, from consumer electronics to industrial equipment.[1]

Working of Microcontroller:

The microcontroller chip is a high-speed device, yet it is slow when compared to a computer. As a result, each command will be executed quickly within the microcontroller. The quartz oscillator is enabled and through control logic register once the supply is powered on. Parasite capacitors will be recharged for a few seconds while the early preparation is taking place. Once the voltage level reaches its maximum value and the oscillator's frequency stabilizes, the operation of writing bits through special function registers becomes stable. Everything is controlled by the oscillator's CLK, and the whole electronics will begin to function. All of this happens in a matter of nanoseconds.[1]

A microcontroller's major role is that it can be thought of as a self-contained system with a processor memory. Its peripherals can be used in the same way that an 8051 microcontroller can. The bulk of microcontrollers in use today are embedded in other types of machinery such as telephones, appliances, vehicles, and computer system peripherals.[1]

Microcontroller Properties :

- Microcontroller devices are capable of having words longer than 64 bits.
- Microcontrollers consist of RAM , ROM , Timer , I/O Ports.
- Microcontroller ROM is used for program storage and RAM is used for data storage.

- It is designed by using CISC architecture.
- The power consumption of modern microcontrollers is significantly lower and have operating voltage range from 1.8V to 5.5V
- The latest feature of microcontroller is flash memory like EPROM and EEPROM.
- The most recent feature of a microcontroller is flash memory, such as EPROM and EEPROM.[1]

Architecture:

The PIC16F877A is based on the Harvard architecture. This architecture separates the memory space for program instructions and data, allowing for efficient and simultaneous access to both. This enhances the performance of the microcontroller.

Key Components of the PIC16F877A:

- CPU: The central processing unit executes instructions and controls the flow of data within the microcontroller.
- Memory: The microcontroller includes both program memory (Flash) and data memory (RAM).
- I/O Ports: These ports allow the microcontroller to interact with external devices.
- Timers: Used for timing events and generating specific time delays.
- Analog-to-Digital Converter (ADC): Converts analog signals (like voltage) into digital values.
- Serial Communication Modules: Support various communication protocols like UART, SPI, and I2C.
- Interrupt System: Enables the microcontroller to respond to external events efficiently.

Supported Protocols:

The PIC16F877A supports several communication protocols, including:

- UART (Universal Asynchronous Receiver-Transmitter): For serial communication with other devices, such as computers or other microcontrollers.
- SPI (Serial Peripheral Interface): For high-speed communication with devices like sensors, displays, and memory chips.
- I2C (Inter-Integrated Circuit): For low-speed communication with multiple devices on a single bus.

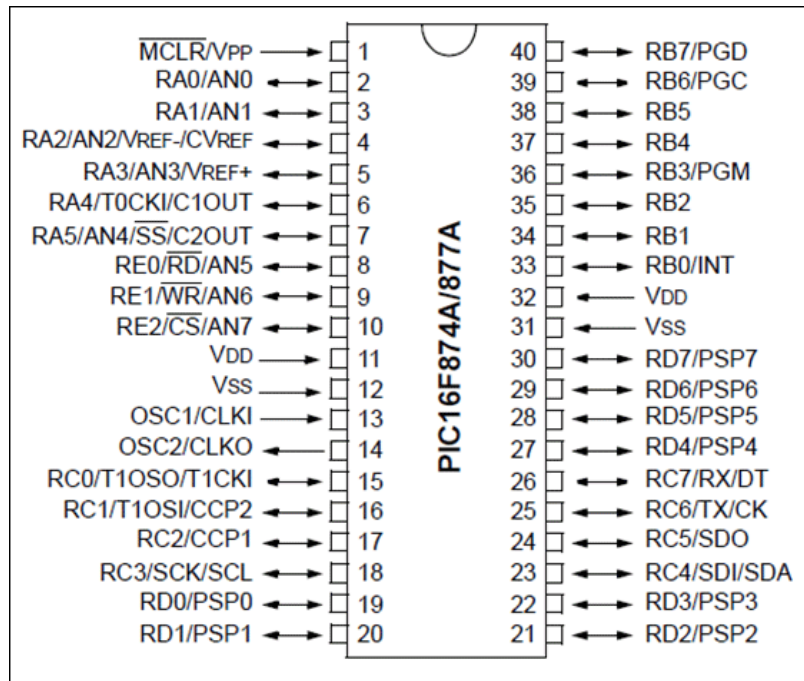


FIGURE 1.0 SHOWING THE PIC16F877A PINOUT

Memory: It contains 368 bytes of RAM (Data Memory) and 256 bytes of EEPROM for non-volatile storage, allowing for both temporary and permanent data storage.

I/O Pins: The PIC16F877A provides 33 input/output (I/O) pins, which can be configured for various purposes, such as digital or analogue inputs, PWM (Pulse Width Modulation) outputs, or serial communication.

- **Port A:** This port is 7-bit wide and can be used for both input and output. The status of TRISA register decided whether it is used as input or output port.
- **Port B:** It is an 8-bit port. This port also can be used as input and output. Moreover in input mode four of its bits are variable according to the interrupt signals.
- **Port C:** It is also an 8-bit port and can be used as both input and output port which is determined by the status of the TRISC register.
- **Port D:** This 8-bit port, unlike Port A, B and C is not an input/output port, but is used as acts as a slave port for the connection to the microprocessor. When in I/O mode Port D all pins should have Schmitt Trigger buffers.
- **Port E:** It is a 3-bit port which is used as the additional feature of the control signals to the A/D converter.

LCD

The full form of LCD is Liquid Crystal Display. it is a passive device, which means that it does not deliver any light to display characters, animations, videos, etc. LCD uses fluorescent tubes to lighten the picture, but can't provide a clearer picture as LED delivers. The LCD 16x2

interface uses a parallel interface, which requires multiple pins to transmit data and control signals. However, some variants also offer an I²C interface, which reduces the number of required pins and is particularly suitable for projects using Arduino or other microcontrollers.[4]

Key Pins

1. RS (Register Select): Selects whether the data sent is an instruction or data.
2. EN (Enable): Enables or disables the LCD module.
3. D4-D7 (Data Pins): Carry the data bits (4-7) for commands and characters.
4. Backlight Pins (optional): Control the backlight, if available.

The LCD operates using two registers:

Command Register: Stores instructions to control the LCD's behavior, such as initialization, screen clearing, and cursor positioning.

Data Register: Stores the ASCII codes of characters to be displayed on the screen.

Summary of LCD Module Control and Data Lines

1. Control Lines (3 total)

- i. EN (Enable): Signals the LCD to receive data. Sequence:
 - Set EN to low (0).
 - Prepare RS, RW, and data bus.
 - Set EN to high (1), wait per the datasheet timing.
 - Set EN to low (0).
 -
- ii. RS (Register Select):
 - Low (0): Data is a command (e.g., clear screen, position cursor).
 - High (1): Data is text for display.
 -
- iii. RW (Read/Write):
 - Low (0): Write to LCD.
 - High (1): Read from LCD (used mainly for "Get LCD status").

2. Data Bus:

-4-bit mode: 7 lines (3 control + 4 data).

-8-bit mode: 11 lines (3 control + 8 data).

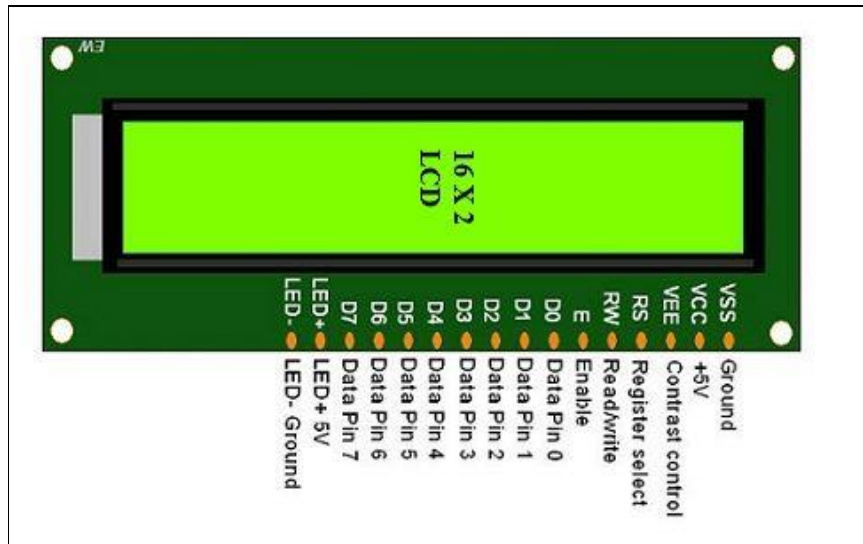


FIGURE 2- DIAGRAM OF LCD 16X2 PINOUT

4X4 KEYPAD

A 4x4 keypad is a matrix-based input device consisting of 16 buttons arranged in 4 rows and 4 columns.

How Does the Matrix Keypad Work?

The matrix keypad consists of pushbutton contacts that are connected to the row and column lines. There is one pin for each column and one pin for each row. So the 4×4 keypad has $4 + 4 = 8$ pins, while the 4×3 keypad has $4 + 3 = 7$ pins.

Keypad Scanning

Here is how a microcontroller scans rows and columns to identify which button has been pressed.

1. Each row is connected to an input pin, and each column is connected to an output pin.
2. Input pins are pulled HIGH by enabling internal pull-up resistors.
3. The microcontroller then sequentially sets the pin for each column LOW and then checks to see if any of the row pins are LOW. Because pull-up resistors are used, the rows will be high unless a button is pressed.
4. If a row pin is LOW, it indicates that the button for that row and column is pressed.
5. The microcontroller then waits for the switch to be released. It then searches the keymap array for the character that corresponds to that button.

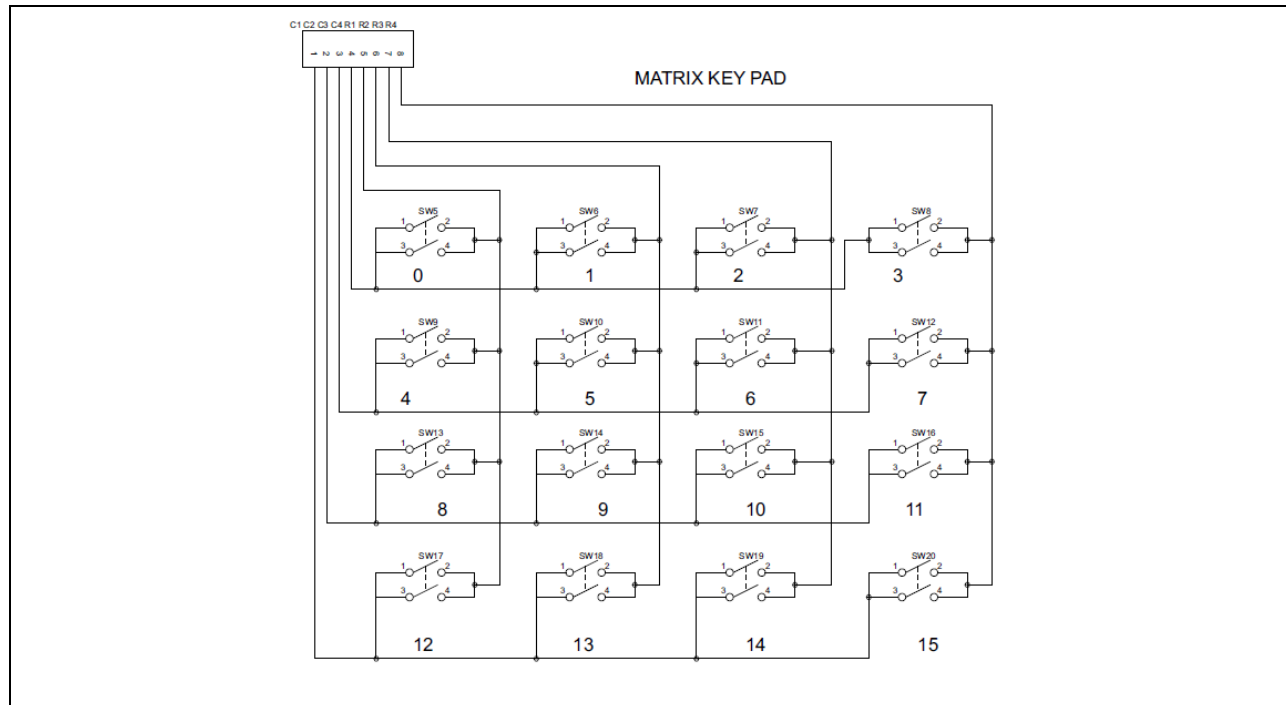


FIGURE 3: DIAGRAM OF THE INTERNAL STRUCTURE OF THE 4X4 MATRIX KEYPAD

OBJECTIVES

- Develop a digital voting system that enables voters to select candidates using a 4x4 keypad.
- Implement a clear and user-friendly interface on an LCD display.
- Ensure accurate and secure vote recording using microcontroller-based logic.
- Utilize LED indicators to provide visual feedback on voting status.
- Incorporate appropriate security measures to protect sensitive information.

METHODOLOGY

Materials and software's used;

- Notepad ++ (software)
- MPLABx IDE (software)
- Proteus software simulation(software)
- PIC16F877A microcontroller
- 20X4 LCD
- 4x4 Keypad
- resistors
- 8MHz crystal
- 2*22pF non polarized capacitors

- 5V power source

PROCEDURE

PROCEDURE FOR SETTING UP THE HEX CODE

Task 1: Creating a project on MPLab x

1. Open MPLab x.
2. Create a new project
3. Select categories: Microchip Embedded and projects: Standalone projects. Click
4. on next.
5. Select the device type as PIC16F877A. Under tools select simulator. You may search for the microcontroller by entering 877A under device.
6. Select the compiler: xc8
7. Give your project a name and folder. Set it as the main project, you must always set the project you are working on as the main project.
8. Click on finish

C CODING

1. Right click on 'Header Files', select new, then select xc8_header.h
2. You may rename you file to anything you desire, but it is better to name it as lcd.h
3. Open lcd.h. You should see something like this:
Figure showing the Header file called lcd.h
4. Copy and paste lcd.h header code lines below. The TAs will explain the code for you.
Next:
5. Right click on 'Source Files', select new, then select main.c
6. You may rename you file to anything you desire, but it is better to name it as main.c
7. Open main.c. You should see something like this:
8. On the file menu, click on 'Production', select 'Set Configuration Bits'

9. Figure showing how to select 'Set Configuration Bits'

10. The FOSC is the the clock that controls the execution of the instructions. Make it external. Turn off everything else.

11. Generate the Source Code to Output.

12. Define the inputs and outputs for the keypad after the LCD at the point:

```
#define RS RD2
```

```
#define EN RD3
```

```
#define D4 RD4
```

```
#define D5 RD5
```

```
#define D6 RD6
```

```
#define D7 RD7
```

```
#define R1 RB4
```

```
#define R2 RB5
```

```
#define R3 RB6
```

```
#define R4 RB7
```

```
#define C1 RB2
```

```
#define C2 RB1
```

```
#define C3 RB0
```

13. Keep the library as '#include <xc.h>'

14. Define the external clock as 8MHz: #define _XTAL_FREQ 8000000

The lab technicians will now help you to write the 'void main()' method. The method allows a user to write characters on the LCD using the PIC microcontroller. Once done, you will clean and build the main project. This will produce a file with an extension called .hex.

Using Proteus to simulate the Hardware

1. Create a new project in the Proteus software.

2. Create the project name, the path where it will be located. Click next.

3. Create a DEFAULT schematic template. Click next.
4. Do not create a PCB layout. Click next.
5. Select 'No Firmware Project'. Click next.
6. Click on 'Finish'.

Key concepts

- setting the oscillator mode for the microcontroller to (HS) because we are using the crystal oscillator which is set to **8MHz**.
- The addition of pulldown resistors on the keypad pins was crucial in preventing noise and signal bouncing. Without these resistors, the inputs from the keypad could result in false triggering, causing the microcontroller to register incorrect key presses. This signal bouncing would also lead to random or erroneous characters being displayed on the LCD. By adding the pulldown resistors, the input signals were stabilized, ensuring accurate key detection and preventing spurious readings, thus improving the overall reliability of the system.
- Debouncing: Implementing pull down resistors to handle signal bouncing caused by mechanical key presses to ensure accurate input.

PSEUDO CODE FOR DIGITAL VOTING SYSTEM

Start

```
Blink LED
Display "Welcome Message"

Enter PIN Code
Input PIN

Check PIN
  If INVALID PIN
    Display "INVALID PIN"
    Go to Enter PIN Code
  Else
    Display "Welcome Message"
Display Candidates
Button Press
Prompt for Vote
Enter Vote
Record Vote
Turn on Confirmation LED
Display "Thanks for Voting"
Display Vote Count
```

Press **I** to Restart Voting

End

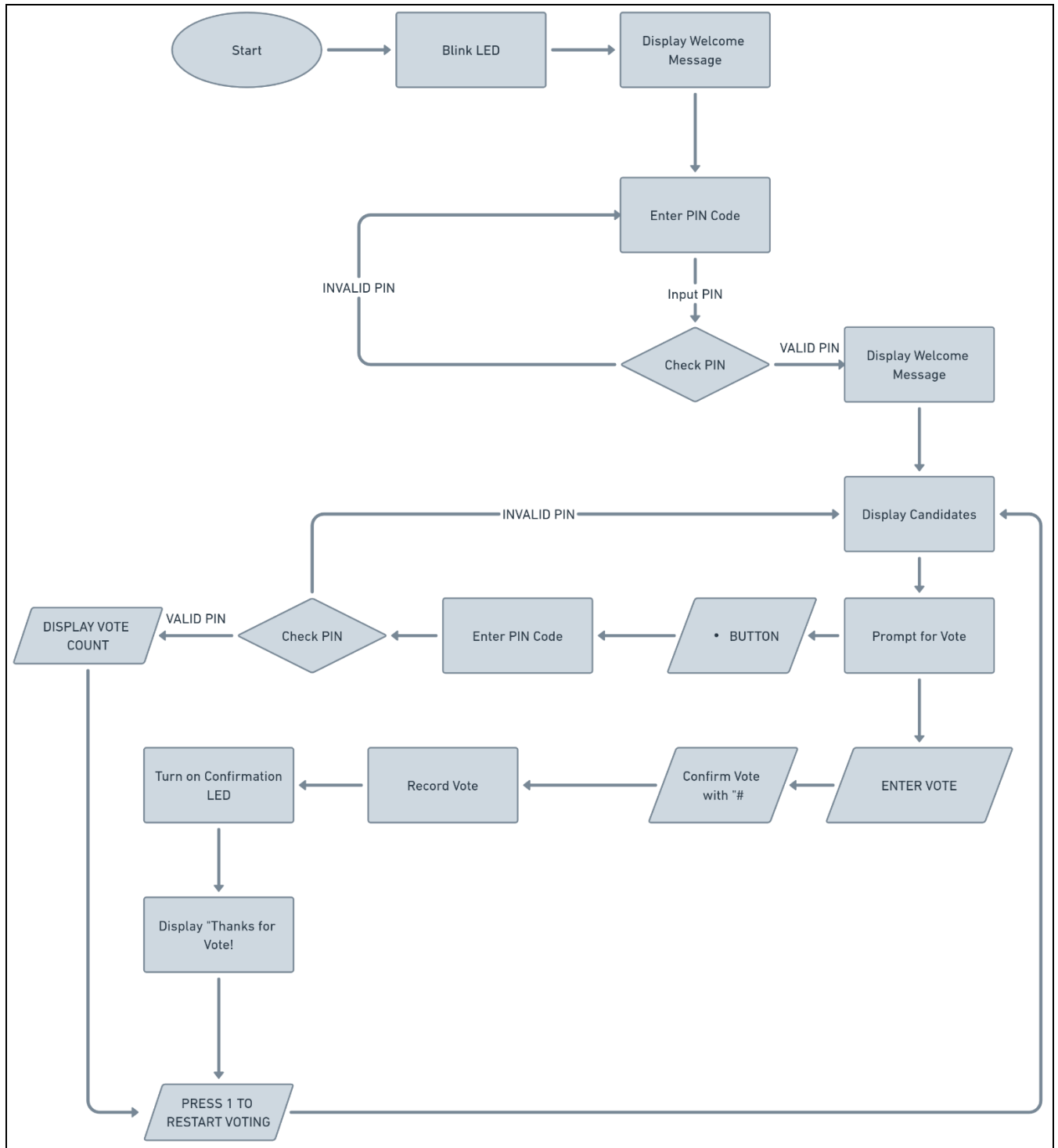


FIGURE 4: FLOWCHART OF THE VOTING SYSTEM

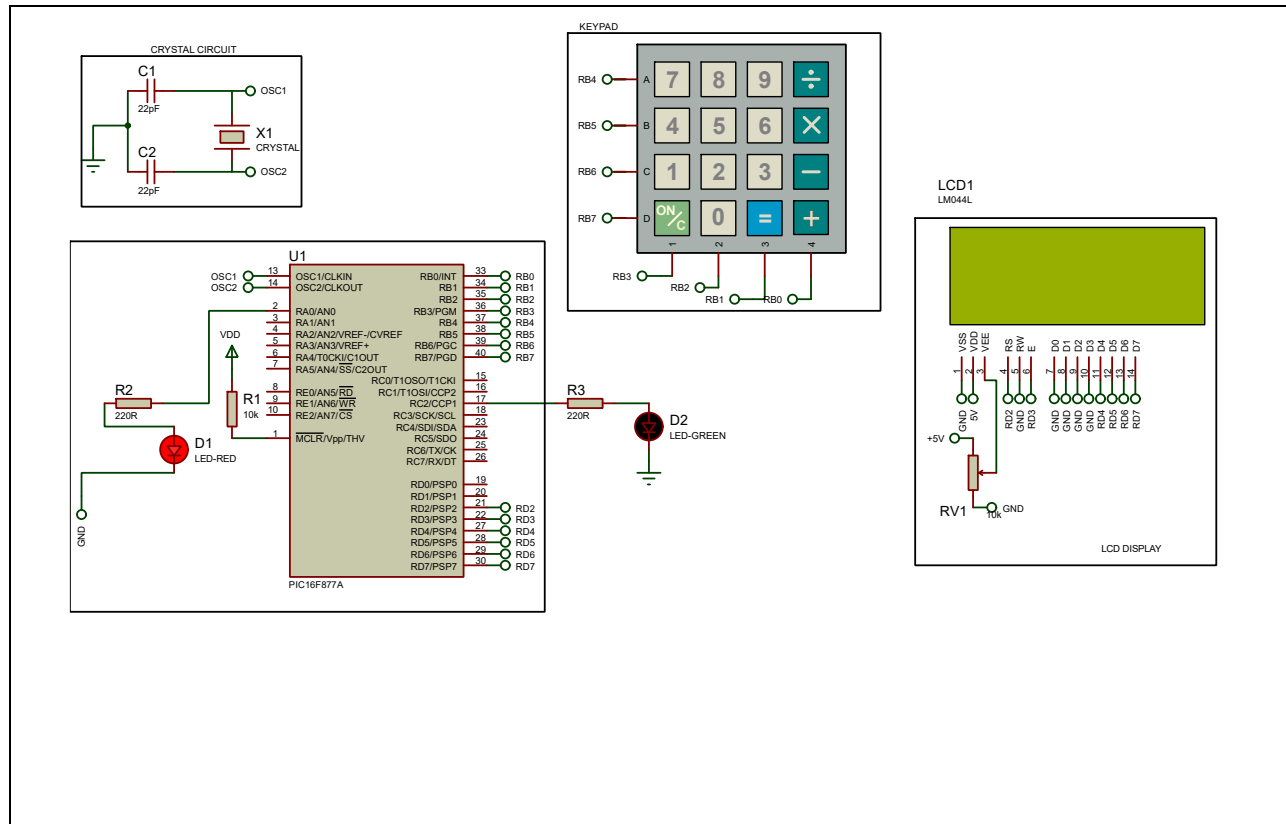


FIGURE 5: SIMULATION DIAGRAM OF THE VOTING SYSTEM

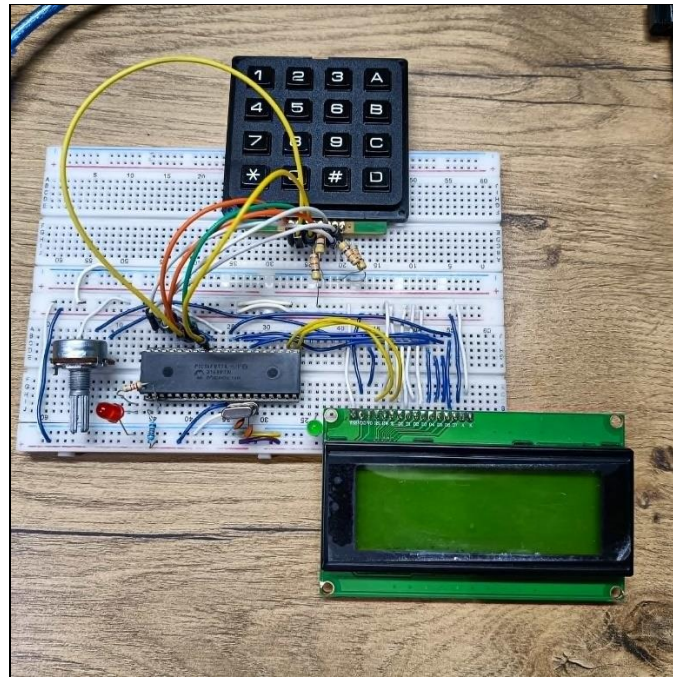


FIGURE 6: BREADBOARD CONNECTIONS OF THE VOTING SYSTEM

RESULTS AND ANALYSIS

- i) Initial power up of digital voting system
- a) simulation

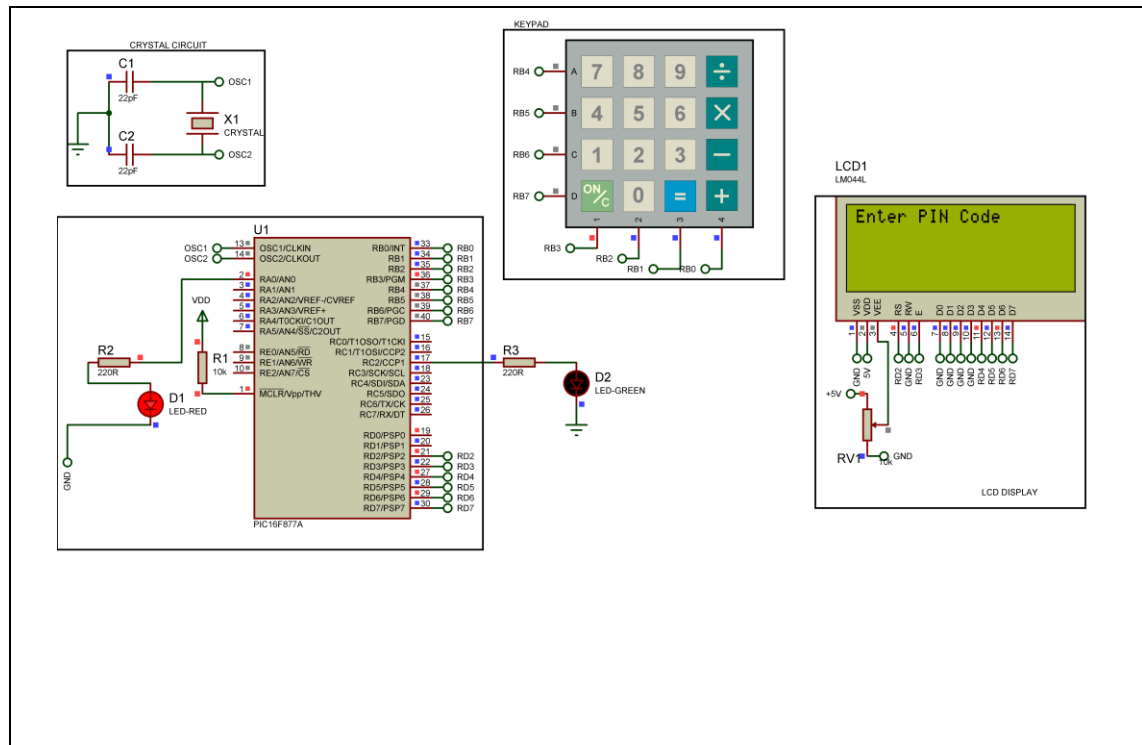


FIGURE 7: DIAGRAM OF INTIAL STATE OF SYSTEM

b) hardware

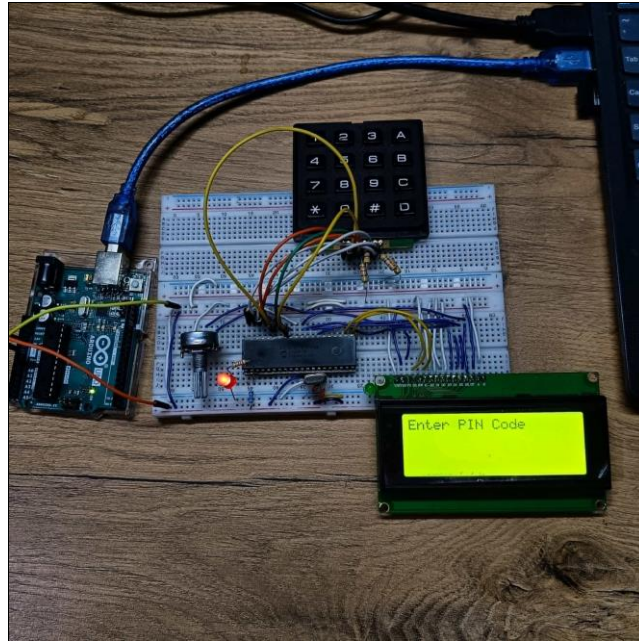


FIGURE 7.1: VOTING DIGITAL SYSTEM AT INTIAL STATE

- When the PIC16F877A is connected to a power source, the **RED** LED will blink 3 times to represent the start up of the **Digital Voting System**
 - After the start up of the Digital Voting System, the **RED** LED will stay on until the system is shut down or switched off.
- ii) *Input of invalid PIN code*
a) *Simulation*

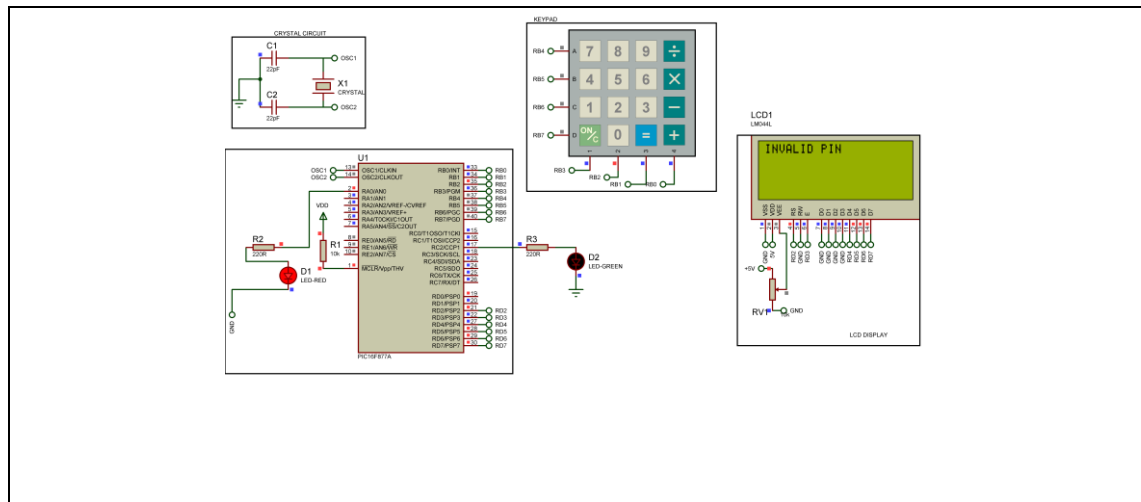


FIGURE 8: OUTPUT OF INVALID PIN COMBINATION

b) hardware

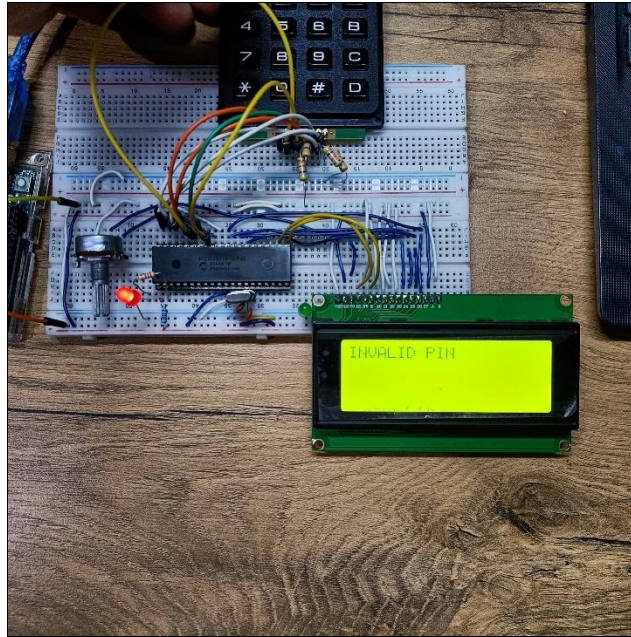


FIGURE 8.1: HARDWARE OUTPUT OF INVALID PIN COMBINATION

- when an invalid PIN is used, the Digital Voting System will not initialize and prompt to enter the PIN again.

iii) *input of valid PIN code*

a) *simulation*

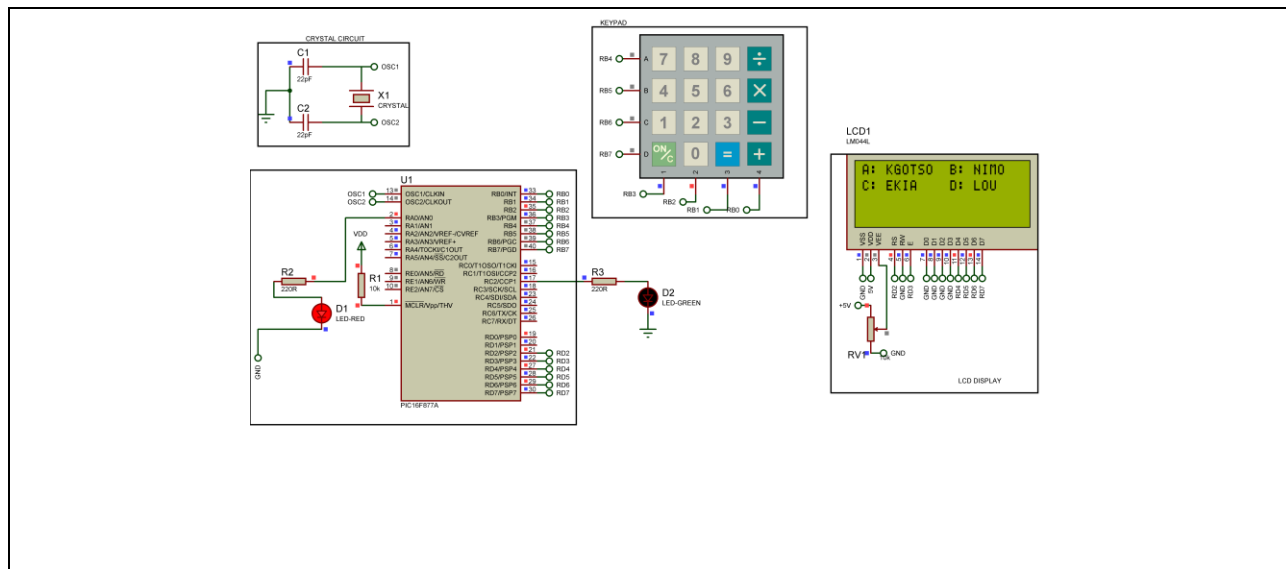


FIGURE 9:OUTPUT OF VALID PIN COMBINATION

b) *hardware*

- when the valid PIN has been inputted, the **Digital Voting System** will display all the election candidates

iv) *waiting for vote input*

a) *simulation*

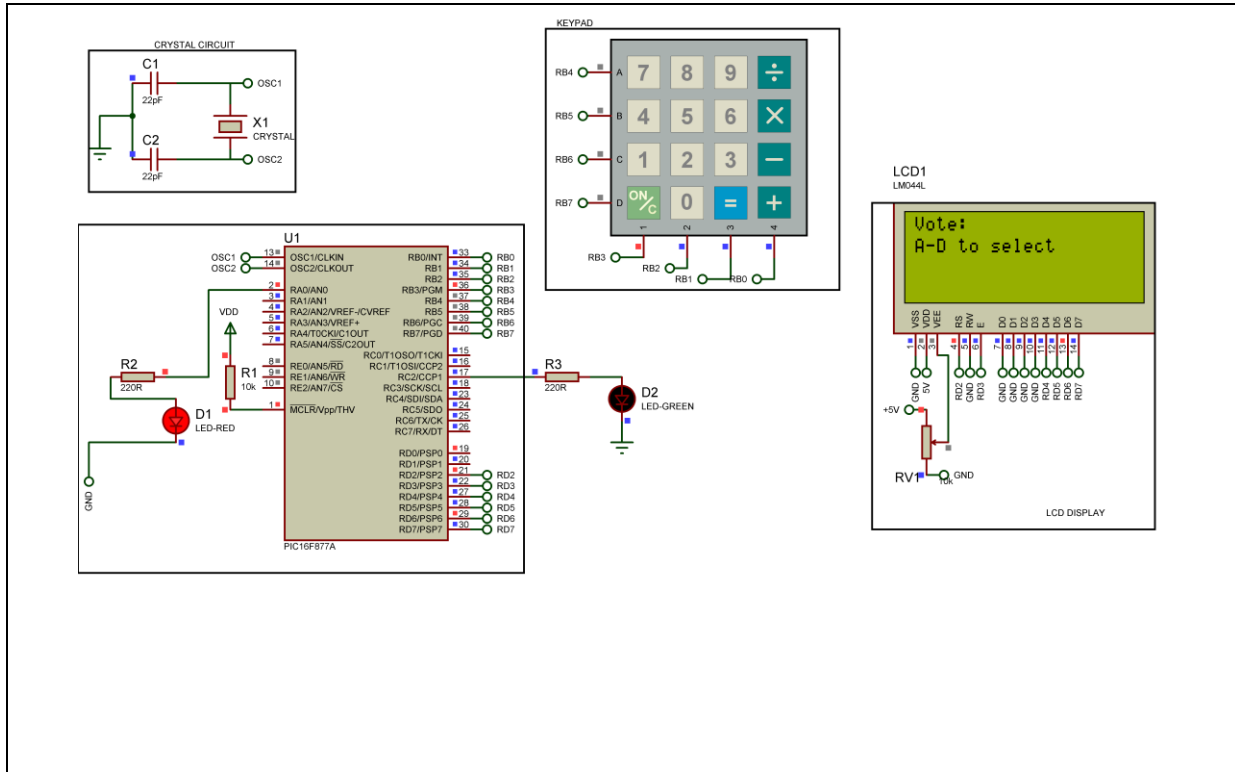


FIGURE 10: STATE OF SYSTEM WAITING FOR VOTE INPUT

b) hardware

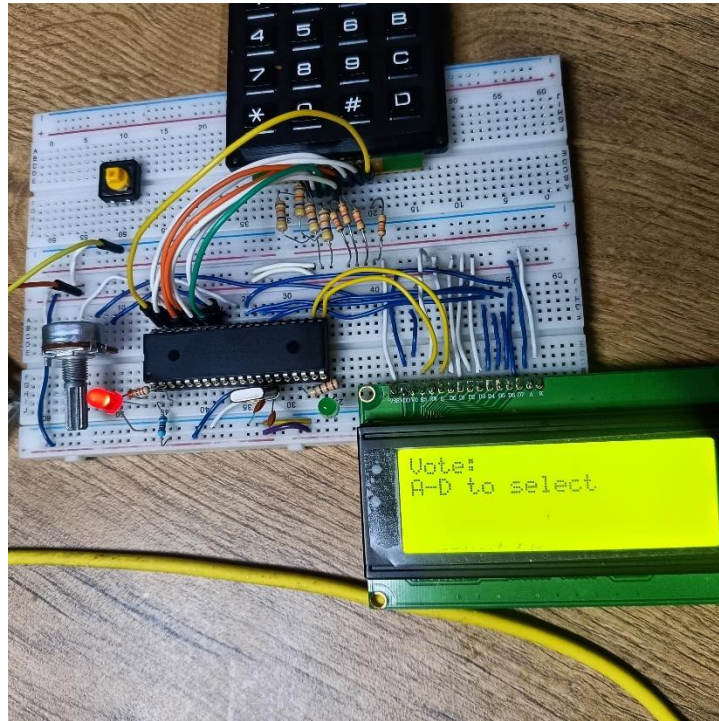


FIGURE 10.1: STATE OF SYSTEM WAITING FOR VOTE INPUT

v) *confirming the vote input*
a) *simulation*

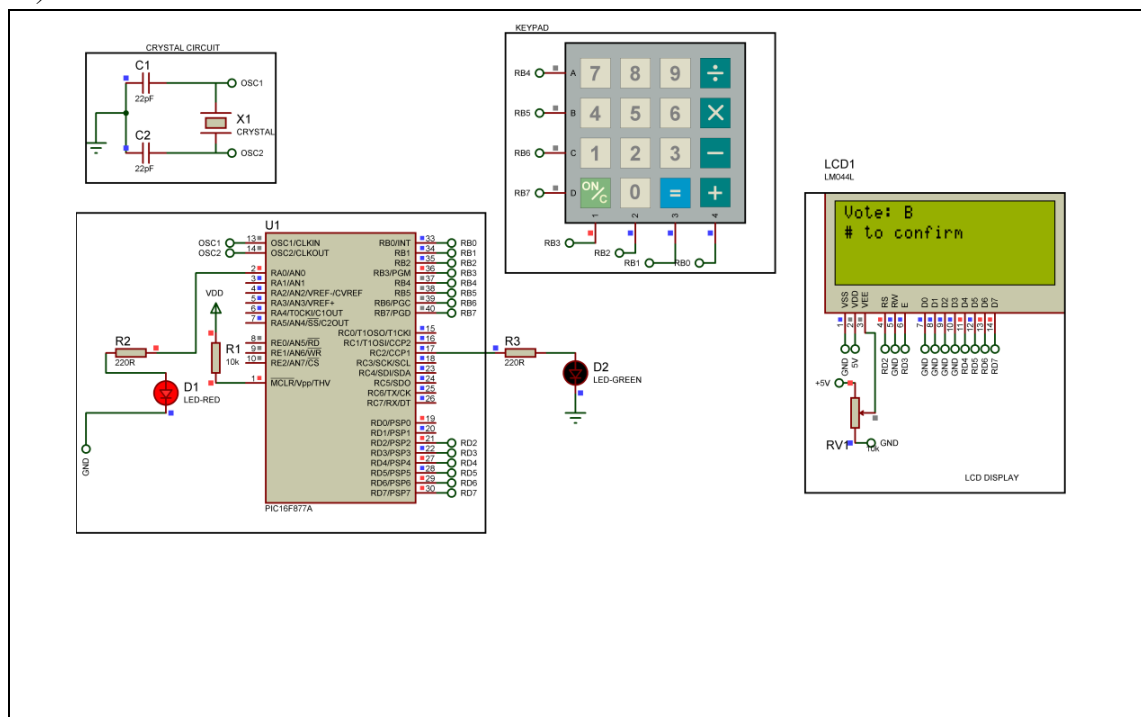


FIGURE 11: CONFIRMING VOTE ON DIGITAL VOTING SYSTEM

b) hardware

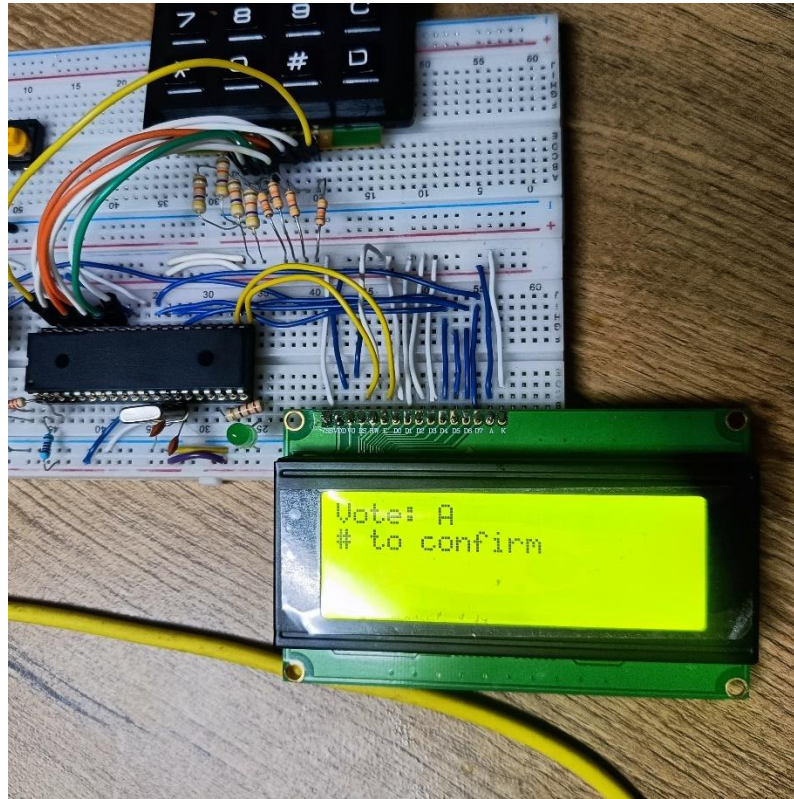


FIGURE 11.1: CONFIRMING VOTE ON DIGITAL VOTING SYSTEM

- Input the choice corresponding with the desired candidate and thereafter, the **Digital Voting System** will prompt you to confirm the vote.

- vi) confirming the vote
a) simulation

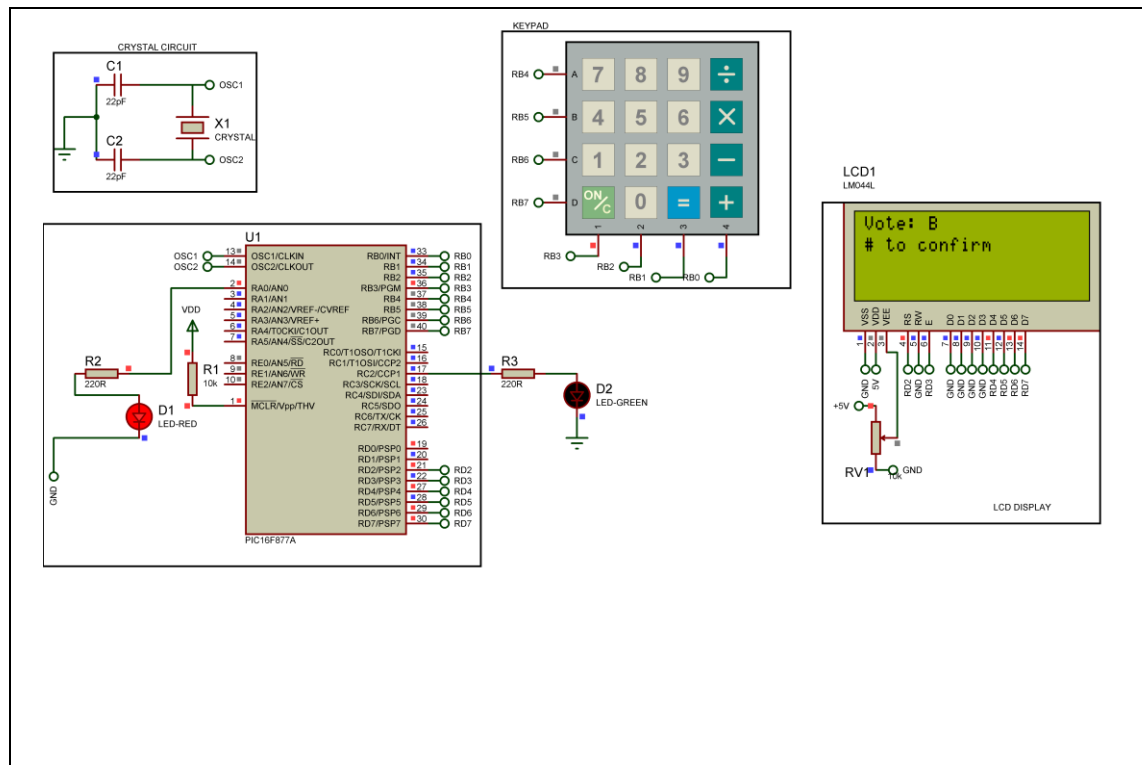


FIGURE 12: CONFIRMING VOTE ON DIGITAL VOTING SYSTEM

- b) hardware

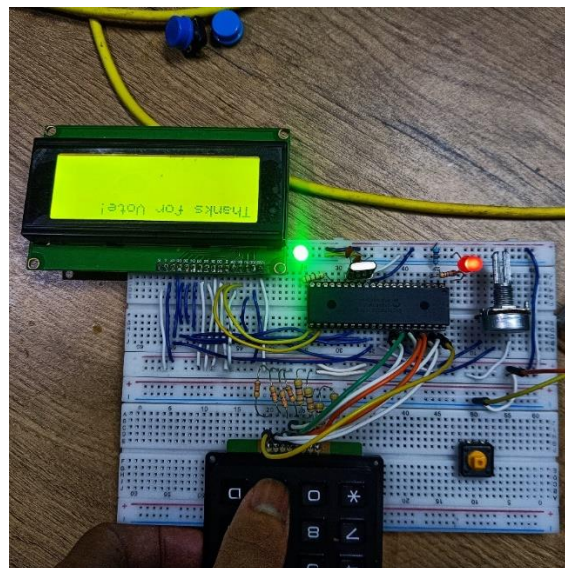


FIGURE 12.1: CONFIRMING VOTE ON DIGITAL VOTING SYSTEM

- after confirming the vote, the GREEN LED will blink to indicate the vote has been casted successfully .

vii) displaying vote statistics

a) simulation

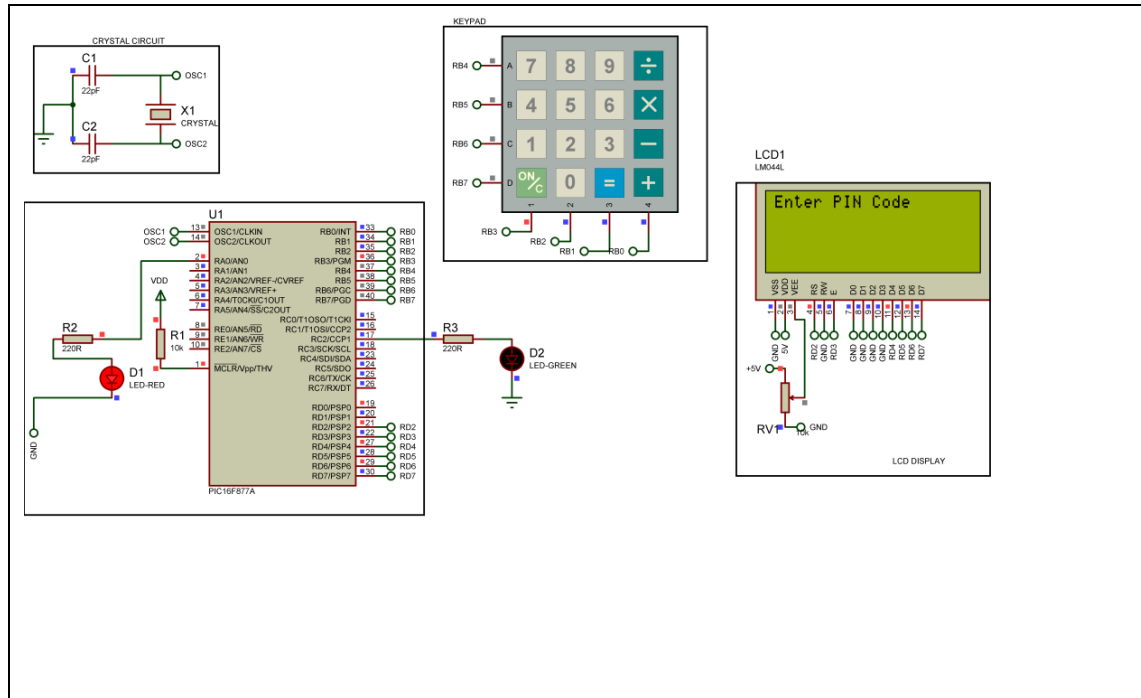


FIGURE 13 : DIAGRAM OF SYSTEM REQUESTING PINCODE TO DISPLAY VOTE TALLY

b) hardware

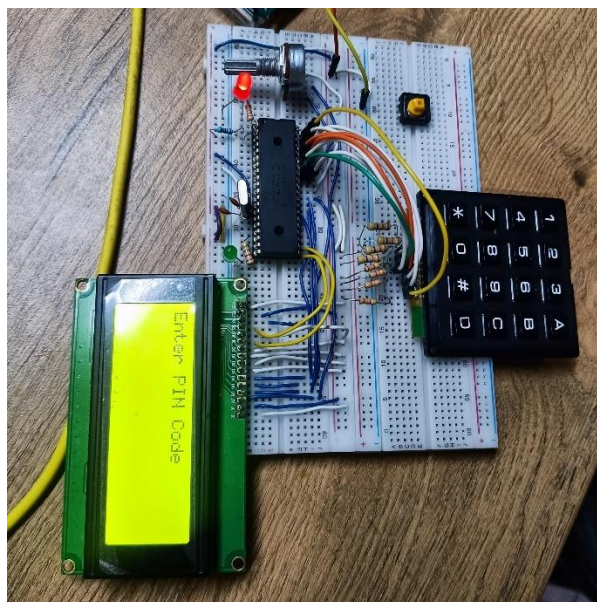


FIGURE 13.1 :DIAGRAM OF SYSTEM REQUESTING PINCODE TO DISPLAY VOTE TALLY

- Pressing the “#” key to display vote statistics will prompt to enter the PIN code to access them.

viii) *entering invalid PIN CODE to display vote statistics*

a) *simulation*

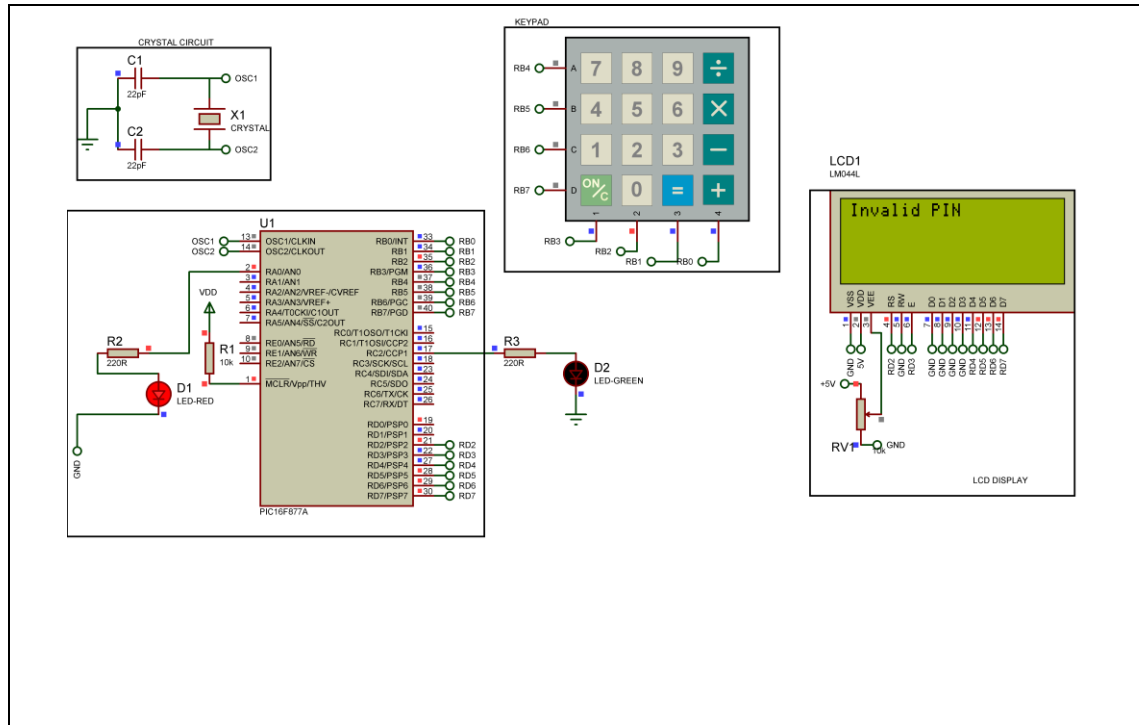


FIGURE 14: OUTPUT OF INPUTTING INVALID CODE

ix) *entering the VALID PIN CODE to display vote statistics*

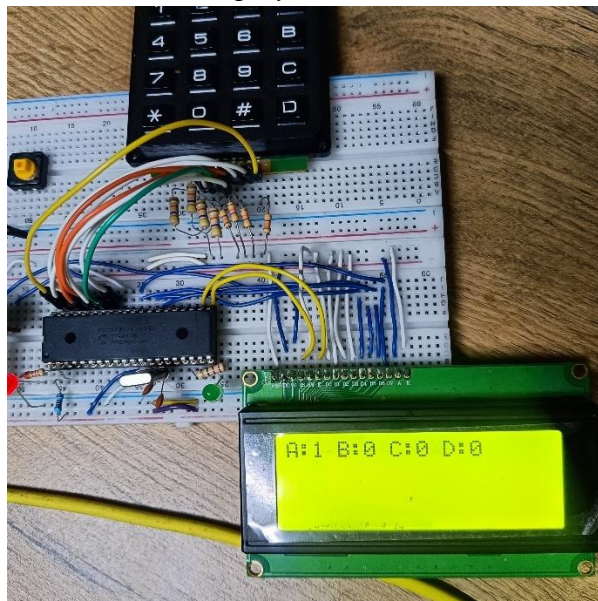


FIGURE 14.1: OUTPUT OF INPUTTING VALID CODE

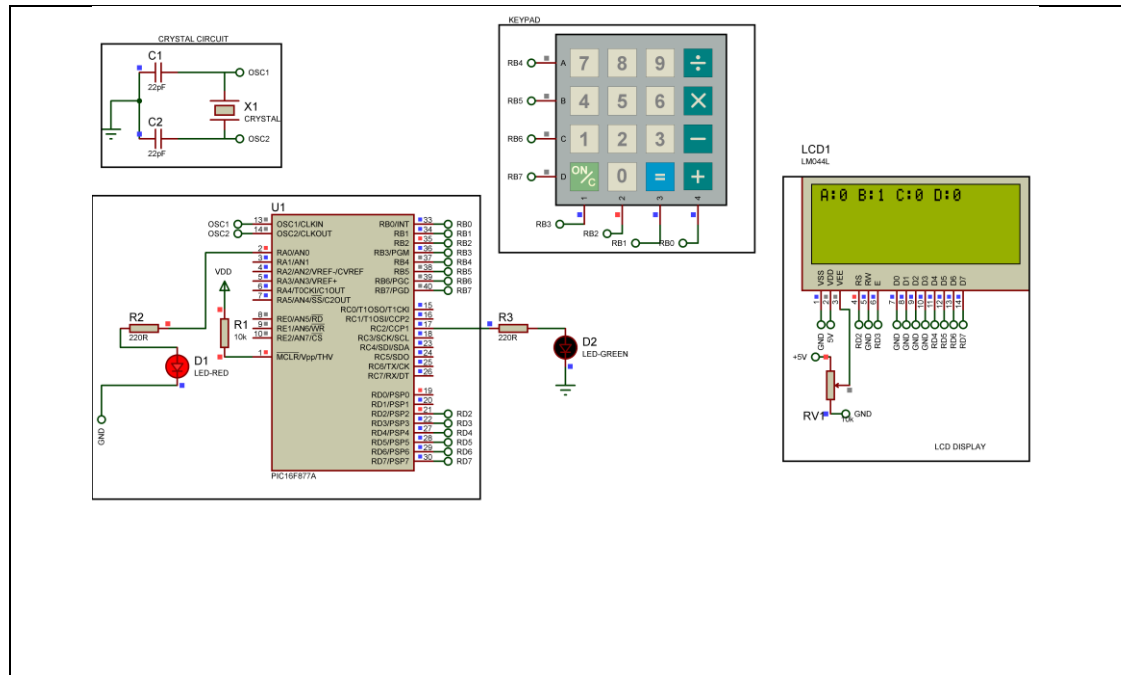


FIGURE 15: SYSTEM SHOWING VOTING STATISTICS

a) *Hardware*

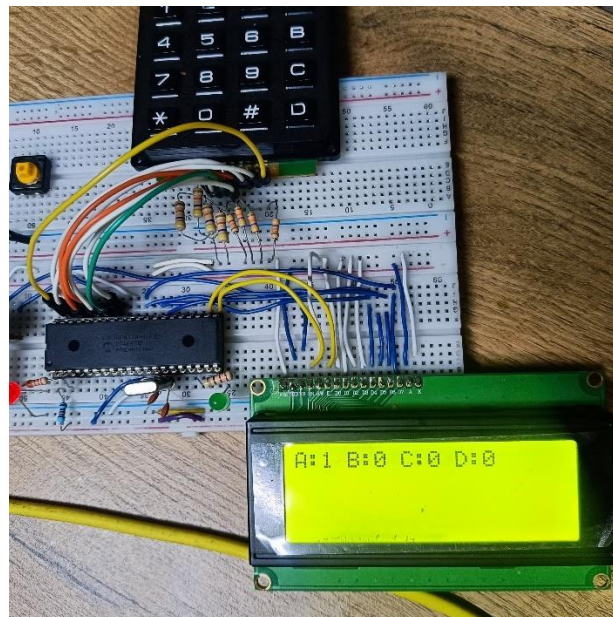


FIGURE 15.1: SYSTEM SHOWING VOTING STATISTICS

- When the correct PIN CODE has been entered, the **Digital Voting System** will display the voting statistics.

DISCUSSION

The digital voting system was successfully implemented, both in simulation and on a physical breadboard. It met all the outlined requirements, demonstrating reliability and user-friendliness. Voters could select candidates by inputting the corresponding letter on a 4x4 keypad. The system's interface, displayed on an LCD, was clear and easy to navigate. To confirm a vote, a green LED blinked, providing immediate visual feedback and enhancing user confidence in the voting process.

Security was a key focus. The system required an administrator PIN during startup and before displaying vote counts, ensuring that sensitive data remained accessible only to authorized personnel. This feature enhanced the system's integrity and helped protect the voting results. Additionally, the system displayed great accuracy, recording votes efficiently and securely.

Despite its success, the project faced some challenges. The microcontroller, operating at a high frequency of 8 MHz, was prone to noise interference. Prolonged power supply caused some microcontroller pins to charge, leading to incorrect data inputs from the keypad. To address this, all keypad pinouts were grounded using 47k Ω resistors, which significantly reduced noise. Although standard practice usually involves grounding only the row pins, grounding all pins proved more effective in mitigating noise and ensuring accurate input.

In terms of future improvements, additional noise filtering could be achieved by adding decoupling capacitors near the microcontroller's power supply pins. Software-based debouncing can further enhance input accuracy by ensuring that only valid key presses are registered. For added reliability, incorporating non-volatile memory would safeguard vote data in case of power interruptions, while redundant data storage could provide an additional layer of security.

CONCLUSION

The successful implementation of the digital voting system demonstrates its potential as a reliable and user-friendly solution for secure voting processes. By integrating a 4x4 keypad for candidate selection, an LCD for a clear interface, and a green LED for vote confirmation, the system ensures a seamless voting experience. Advanced security measures, including administrator PIN protection, further enhance the integrity and confidentiality of the voting process.

Despite initial challenges with noise interference and microcontroller pin charging, effective countermeasures such as grounding all keypad pinouts with 47k Ω resistors were implemented, ensuring accurate data input. These solutions highlight the robustness and adaptability of the system design.

Moving forward, the system can be improved through additional noise filtering, software-based debouncing, and the incorporation of non-volatile memory for data protection. Such enhancements will not only increase the system's reliability but also prepare it for more extensive applications in larger-scale voting scenarios. Ultimately, this project lays a strong foundation for future advancements in secure digital voting technologies.

REFERENCES

- [1] "Microcontroller and its Types," *GeeksforGeeks*, [Online]. Available: <https://www.geeksforgeeks.org/microcontroller-and-its-types/>. [Accessed: Nov. 15, 2024].
- [2] "Microcontroller Architecture," *EDUCBA*, [Online]. Available: <https://www.educba.com/microcontroller-architecture/>. [Accessed: Nov. 15, 2024].
- [3] Microchip Technology Inc., "PIC16F87XA Data Sheet: 28/40/44-Pin Enhanced Flash Microcontrollers," DS39582E datasheet, 2013. [Online]. Available: <https://ww1.microchip.com/downloads/en/devicedoc/39582e.pdf>
- [4] Raystar Optonics, "16x4 Character LCD Display Module," [Online]. Available: <https://www.raystar-optonics.com/character-lcd-display-module/lcd-16x4.html>. [Accessed: Nov. 1, 2024].
- [5] Elektronica Voor Jou, "4x4 Matrix Keypad - Zwart," [Online]. Available: <https://elektronicavoorjou.nl/en/product/4x4-matrix-keypad-zwart/>. [Accessed: 10-Nov-2024].
- [6] Last Minute Engineers, "A Beginner's Guide to Keypad Module for Arduino," *Last Minute Engineers*. [Online]. Available: <https://lastminuteengineers.com/arduino-keypad-tutorial/>. [Accessed: Nov. 10, 2024].
- [7] K. Mathangwane, "Keypad Control System for Microcontroller Project," Unpublished Lab Work, Botswana International University of Science and Technology, Botswana, Nov. 2024.
- [8] T. Seiphepi, "Keypad Control System for Microcontroller Project," Unpublished Lab Work, Botswana International University of Science and Technology, Botswana, Nov. 2024.

APPENDIX

LCD HEADER FILE

```
/*
 * File:
 * Author:KGOTSO MATHANGWANE AND TSOTLHE SEIPHEPI
 * Comments: WE HAD FUN DOING THIS
 * Revision history:
 */

// This is a guard condition so that contents of this file are not included
// more than once.
#ifndef XC_HEADER_TEMPLATE_H
#define XC_HEADER_TEMPLATE_H

#include <xc.h> // include processor files - each processor file is guarded.

// TODO Insert appropriate #include <>

// TODO Insert C++ class definitions if appropriate

// TODO Insert declarations

// Comment a function and leverage automatic documentation with slash star star
/**
    <p><b>Function prototype:</b></p>

    <p><b>Summary:</b></p>

    <p><b>Description:</b></p>

    <p><b>Precondition:</b></p>

    <p><b>Parameters:</b></p>

    <p><b>Returns:</b></p>

    <p><b>Example:</b></p>
    <code>

    </code>

    <p><b>Remarks:</b></p>
 */
// TODO Insert declarations or function prototypes (right here) to leverage
// live documentation

#ifdef __cplusplus
extern "C" {
#endif /* __cplusplus */

    // TODO If C++ is being used, regular C code needs function names to have C
    // linkage so the functions can be used by the c code.
```

```

#ifdef __cplusplus
}
#endif /* __cplusplus */

#endif /* XC_HEADER_TEMPLATE_H */

void Lcd_Port(char a)
{
    if(a & 1)
        D4 = 1;
    else
        D4 = 0;
    if(a & 2)
        D5 = 1;
    else
        D5 = 0;
    if(a & 4)
        D6 = 1;
    else
        D6 = 0;
    if(a & 8)
        D7 = 1;
    else
        D7 = 0;
}

void Lcd_Cmd(char a)
{
    RS = 0; // => RS = 0
    Lcd_Port(a);
    EN = 1; // => E = 1

    EN = 0; // => E = 0
}

void Lcd_Clear()
{
    Lcd_Cmd(0);
    Lcd_Cmd(1);
}

void Lcd_Set_Cursor(char a, char b)
{
    char temp,z,y;
    if(a == 1)
    {
        temp = 0x80 + b - 1;
        z = temp>>4;
        y = temp & 0x0F;
        Lcd_Cmd(z);
        Lcd_Cmd(y);
    }
    else if(a == 2)
    {
        temp = 0xC0 + b - 1;
        z = temp>>4;
        y = temp & 0x0F;
        Lcd_Cmd(z);
    }
}

```

```

    Lcd_Cmd(y);
}
}
void Lcd_Init()
{
    Lcd_Port(0x00);

    Lcd_Cmd(0x03);

    Lcd_Cmd(0x03);

    Lcd_Cmd(0x03);
    //////////////////////////////////////
    Lcd_Cmd(0x02);
    Lcd_Cmd(0x02);
    Lcd_Cmd(0x08);
    Lcd_Cmd(0x00);
    Lcd_Cmd(0x0C);
    Lcd_Cmd(0x00);
    Lcd_Cmd(0x06);
}
void Lcd_Write_Char(char a)
{
    char temp,y;
    temp = a&0x0F;
    y = a&0xF0;
    RS = 1; // => RS = 1
    Lcd_Port(y>>4); //Data transfer
    EN = 1;

    EN = 0;
    Lcd_Port(temp);
    EN = 1;

    EN = 0;
}
void Lcd_Write_String(char *a)
{
    int i;
    for(i=0;a[i]!='\0';i++)
        Lcd_Write_Char(a[i]);
}
void Lcd_Shift_Right()
{
    Lcd_Cmd(0x01);
    Lcd_Cmd(0x0C);
}
void Lcd_Shift_Left()
{
    Lcd_Cmd(0x01);
    Lcd_Cmd(0x08);
}

```

MAIN SOURCE CODE

```
#define _XTAL_FREQ 8000000

// PIC16F877 Configuration Bit Settings
#pragma config FOSC = HS      // Oscillator Selection bits (XT oscillator)
#pragma config WDTE = OFF     // Watchdog Timer Enable bit (WDT disabled)
#pragma config PWRTE = OFF    // Power-up Timer Enable bit (PWRT disabled)
#pragma config CP = OFF       // Code Protection bit (Code protection off)
#pragma config BOREN = OFF    // Brown-out Reset Enable bit (BOR disabled)
#pragma config LVP = OFF      // Low Voltage ICSP disabled
#pragma config CPD = OFF      // Data EE Memory Code Protection (Code protection off)
#pragma config WRT = OFF      // FLASH Program Memory Write Enable (Unprotected program memory)

// LCD Pin Mapping
#define RS RD2
#define EN RD3
#define D4 RD4
#define D5 RD5
#define D6 RD6
#define D7 RD7

#include <xc.h>
#include "lcd.h"
#include <stdio.h>
#include <string.h>

// Vote counts for candidates
int ttlVotesOf_A = 0; // Candidate A
int ttlVotesOf_B = 0; // Candidate B
int ttlVotesOf_C = 0; // Candidate C
int ttlVotesOf_D = 0; // Candidate D

char votes[16];
char selectedCandidate = '\0';

// Predefined PIN code
const char correctPin[5] = {'8', '0', '0', '8', '5'};
char enteredPin[5];
int pinIndex = 0;

// Keypad matrix mapping
char read_keypad() {
    for (int row = 0; row < 4; row++) {
        // Activate one row at a time
        PORTB = (1 << row);

        if (PORTBbits.RB4 == 1) return (row == 0) ? 'A' : (row == 1) ? '3' : (row == 2) ? '2' : '1';
        if (PORTBbits.RB5 == 1) return (row == 0) ? 'B' : (row == 1) ? '6' : (row == 2) ? '5' : '4';
        if (PORTBbits.RB6 == 1) return (row == 0) ? 'C' : (row == 1) ? '9' : (row == 2) ? '8' : '7';
        if (PORTBbits.RB7 == 1) return (row == 0) ? 'D' : (row == 1) ? '#' : (row == 2) ? '0' : '*';
    }
    return '\0'; // No key pressed
}

void display_candidates() {
```

```

// Display candidate information
Lcd_Clear();
Lcd_Set_Cursor(1, 1);
Lcd_Write_String("A: KGOTSO B: NIMO");
Lcd_Set_Cursor(2, 1);
Lcd_Write_String("C: EKIA D: LOU");
__delay_ms(1500);
Lcd_Clear();
}

void enter_pin() {
    Lcd_Clear();
    Lcd_Set_Cursor(1, 1);
    Lcd_Write_String("Enter PIN Code");

    pinIndex = 0; // Reset PIN input index
    while (pinIndex < 5) {
        char key = read_keypad();
        if (key >= '0' && key <= '9') {
            enteredPin[pinIndex] = key;
            pinIndex++;
            Lcd_Set_Cursor(2, pinIndex);
            Lcd_Write_String("*"); // Show asterisks for entered PIN
        }
        __delay_ms(200); // Debouncing delay
    }
}

void blink_led() {
    for (int i = 0; i < 3; i++) {
        PORTAbits.RA0 = 1; // Turn on LED
        __delay_ms(500);
        PORTAbits.RA0 = 0; // Turn off LED
        __delay_ms(500);
    }
}

void welcome_message() {
    Lcd_Clear();
    Lcd_Set_Cursor(1, 1);
    Lcd_Write_String(" VOTING SYSTEM ");
    __delay_ms(3000);
}

void wait_for_restart() {
    Lcd_Clear();
    Lcd_Set_Cursor(1, 1);
    Lcd_Write_String("Press 1 to");
    Lcd_Set_Cursor(2, 1);
    Lcd_Write_String("Restart Voting");

    while (1) {
        char key = read_keypad();
        if (key == '1') {
            welcome_message();
            break;
        }
    }
}

```

```

    }
    __delay_ms(200); // Debouncing delay
}
}

void main(void) {
    // Configure ports
    TRISA = 0x00; // Set PORTA as output (RA0 for LED)
    TRISB = 0xF0; // RB0-RB3 as outputs (rows), RB4-RB7 as inputs (columns)
    TRISD = 0x00; // Port D as output (LCD)
    TRISC = 0x00; // Port C as output (Confirmation LED)
    PORTA = 0x00; // Clear Port A
    PORTB = 0x00; // Clear Port B
    PORTC = 0x00; // Clear Port C

    // Blink LED 3 times before starting the system
    blink_led();

    // Keep LED on while the system is running
    PORTAbits.RA0 = 1;

    // Initialize LCD
    Lcd_Init();

    // Initial welcome message
    welcome_message();

    while (1) {
        // Loop for PIN entry
        int pinCorrect = 0;
        while (!pinCorrect) {
            enter_pin(); // Prompt for PIN entry

            // Check if entered PIN matches the correct PIN
            if (memcmp(enteredPin, correctPin, 5) == 0) {
                Lcd_Clear();
                Lcd_Set_Cursor(1, 1);
                Lcd_Write_String("WELCOME");
                __delay_ms(1000);
                pinCorrect = 1; // Exit loop if PIN is correct
            } else {
                Lcd_Clear();
                Lcd_Set_Cursor(1, 1);
                Lcd_Write_String("INVALID PIN");
                __delay_ms(1000); // Show invalid PIN message
            }

            // Clear entered PIN after each attempt
            memset(enteredPin, 0, 5); // Reset the enteredPin array
            pinIndex = 0; // Reset the pinIndex
        }

        // Voting process
        while (1) {
            display_candidates();

```

```

// Prompt user to vote
Lcd_Clear();
Lcd_Set_Cursor(1, 1);
Lcd_Write_String("Vote: ");
Lcd_Set_Cursor(2, 1);
Lcd_Write_String("A-D to select");

while (1) {
    char key = read_keypad();

    if (key == 'A' || key == 'B' || key == 'C' || key == 'D') {
        selectedCandidate = key;

        // Confirm vote
        Lcd_Clear();
        Lcd_Set_Cursor(1, 1);
        char message[16];
        sprintf(message, "Vote: %c", selectedCandidate);
        Lcd_Write_String(message);
        Lcd_Set_Cursor(2, 1);
        Lcd_Write_String("# to confirm");
    } else if (key == '#') {
        if (selectedCandidate != '\0') {
            // Record vote
            if (selectedCandidate == 'A') ttlVotesOf_A++;
            else if (selectedCandidate == 'B') ttlVotesOf_B++;
            else if (selectedCandidate == 'C') ttlVotesOf_C++;
            else if (selectedCandidate == 'D') ttlVotesOf_D++;

            // Acknowledge vote
            Lcd_Clear();
            Lcd_Set_Cursor(1, 1);
            Lcd_Write_String("Thanks for Vote!");
            PORTCbits.RC2 = 1; // Confirmation LED
            delay_ms(2000);
            PORTCbits.RC2 = 0; // Turn off LED
            wait_for_restart(); // Wait for restart signal
            break; // Exit to restart voting process
        } else {
            // No candidate selected
            Lcd_Clear();
            Lcd_Set_Cursor(1, 1);
            Lcd_Write_String("Invalid Response");
            delay_ms(1000);
            break; // Exit invalid phase to restart process
        }
    } else if (key == '*') { // Display total votes with PIN entry
        Lcd_Clear();
        Lcd_Set_Cursor(1, 1);
        Lcd_Write_String("Enter Vote PIN");
        enter_pin();

        if (memcmp(enteredPin, correctPin, 5) == 0) {
            Lcd_Clear();
            Lcd_Set_Cursor(1, 1);
            sprintf(votes, "A:%d B:%d C:%d D:%d", ttlVotesOf_A, ttlVotesOf_B, ttlVotesOf_C, ttlVotesOf_D);

```



```
Lcd_Write_String(votes);
__delay_ms(5000); // Display for 2 seconds
wait_for_restart();
break;
} else {
    Lcd_Clear();
    Lcd_Set_Cursor(1, 1);
    Lcd_Write_String("Invalid PIN");
    __delay_ms(1000);
    break;
}

memset(enteredPin, 0, 5); // Reset the enteredPin array
pinIndex = 0; // Reset the pinIndex
} else if (key != '\0') { // Invalid key
    Lcd_Clear();
    Lcd_Set_Cursor(1, 1);
    Lcd_Write_String("Invalid Response");
    __delay_ms(1000);
    break; // Exit invalid phase to restart process
}
__delay_ms(200); // Debouncing delay
}
}
}
```